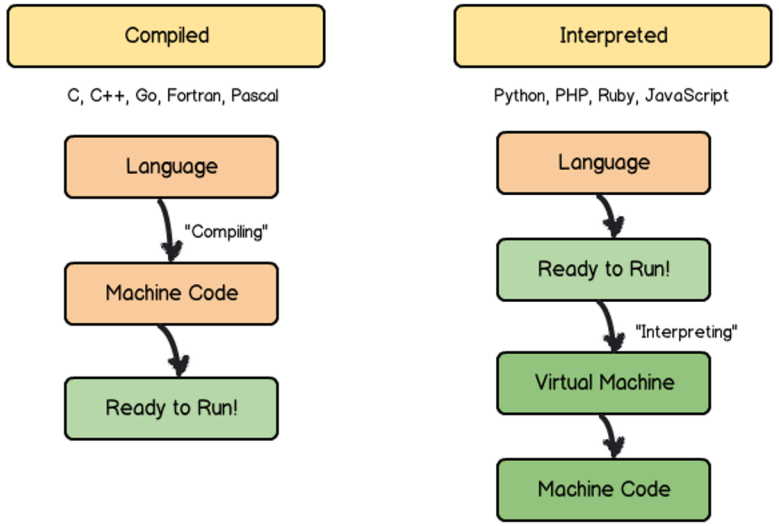
C/C++ vs. Python

Department of Data Science

C/C++ vs. Python

- Differences



	C/C++	Python
Compilation	Compiled language Need compilation	Interpreted language Use interpreter
Code	More lines of code	Less lines of code
Memory Management	No garbage collection	Support garbage collection
Type	Statically typed	Dynamically typed
Speed	Faster	Slower (it uses interpreter and determines the data type at run time)

C and C++

Classes

```
#include <iostream>
using namespace std;
class Person {
  // all members are private by default
  static const int NAME_SIZE = 100;
  int id;
  char name[NAME_SIZE];
public:
  void aboutMe() {
    cout << "I am a person." << endl;
  }
};
```

- By default, all data members and methods are private in C++.
- You can modify this by introducing the keyword public.

Inheritance

- Base (parent) class: Person
- Derived (child) class: Student
- Make sure to delete allocated memory!

```
class Student: public Person {
public:
  void aboutMe() {
    cout << "I am a student." << endl;
  }
};
int main() {
  Student* p = new Student();
  p->aboutMe(); // "I am a student."
  delete p;
  return 0;
}
```

Constructors

```
Person(int a) {
  id = a;
}
```

- Constructor is automatically called upon an object's creation.
- If no constructor is defined, the compiler automatically generates one called the default constructor.
- Alternatively, we can define our own constructor.

```
Person(int a): id(a) {


}
```

- The member id is assigned before the actual object is created and before the remainder of the constructor code is called.
  - When fields are constant/class types.

Destructors

```
~Person() {
  // Free any memory allocated
  // within class
  delete obj;
}
```

- Clean up upon object deletion and is automatically called when an object is destroyed
- Cannot take an argument as we don't explicitly call a destructor.

Virtual Functions

- What happen if we define p to be a Person* ?
- "I am a person."
- Function aboutMe is resolved at compile time in a mechanism static binding.

```
class Student: public Person {
public:
  void aboutMe() {
    cout << "I am a student." << endl;
  }
};
int main() {
  Person* p = new Student();
  p->aboutMe();
  delete p;
  return 0;
}
```

Virtual Functions

- What if you want to ensure that aboutMe of the Student class is called?
- Define aboutMe in the Person class to be virtual.

```
class Person {
  ...
  virtual void aboutMe() {
    cout << "I am a person." << endl;
  }
};
```

```
class Student: public Person {
public:
  void aboutMe() {
    cout << "I am a student." <<
endl;
  }
};
```

Virtual Functions

```
class Shape {
  double width;
  double height;
  virtual double area() = 0;
};
class Rectangle: public Shape {
public:
  double area() {
    return (width * height);
  }
};
class Triangle: public Shape {
public:
  double area() {
    return (width * height)/2;
  }
};
```

- When we don't want to implement a method for the parent class
  - Rectangle and Triangle inherit from Shape so we can implement a common method `addCourse(string s)`.
  - Calling `addCourse` in Shape would not make sense.

```
int main() {
  Shape* s = new Rectangle();
  cout<<s->area()<<endl;
  delete s;
}
```

Virtual Functions

- By defining addCourse to be a "pure virtual function", Person is an abstract class and we cannot instantiate it.

```
class Person {
  ...
  virtual void aboutMe() {
    cout << "I am a person." << endl;
  }
  virtual bool addCourse(string s) = 0;
};
```

```
int main() {
  Person* p = new Person(); // error
  p->aboutMe(); // "I am a student."
  p->addCourse("Data Structures");
  delete p;
}
```

Default Values

- Functions specify default values.
- All default parameters must be on the right side of the function declaration.

```
int func(int a, int b = 3) {
  int x = a;
  int y = b;
  return a + b;
}

w = func(4);
z = func(4, 5);
```

Pointers

- Hold the address of a variable
- Used to perform any operation directly done on the variable
  - Accessing the variable
  - Modifying the variable
- Two pointers can equal each other
- The size of a pointer varies depending on the architecture
  - 32 bits on a 32-bit machine
  - 64 bits on a 64-bit machine
- Output

  8

```
int* p = new int;
*p = 7;
int* q = p;
*p = 8;
cout << *q; // ?
```

References

- Another name (alias) for a pre-existing object.
- Do not have memory of its own.
- Output

  7

```
int a = 5;
// b is reference to a
int& b = a;
// modifying b will also modify a
b = 7;
cout << a; // ?
```

References

- You cannot create a reference without specifying where in memory it refers to.
- But you can create a free-standing reference.
- Unlike pointers
  - reference cannot be null
  - reference cannot be reassigned to another piece of memory

```
// Allocate memory to store 12
// and make b a reference to
// this piece of memory
const int& b = 12;
```

Pointer Arithmetic

- Performing p++ skips ahead by sizeof(int) bytes.
- Output

  1
- Had p been of different type, it would skip ahead as many bytes as the size of the data structure.

```
int* p = new int[2];
p[0] = 0;
p[1] = 1;
p++;
cout << *p; // ?
```

Templates

- A way of reusing code to apply the same class to different data types
- Goal: list-like data structure which we would like to use for lists of various types

```
Template <class T> class ShiftedList {
  T* array;
  int offset, size;
public:
  ShiftedList(int sz):offset(0),size(sz){
    array = new T[size];
  }
  ~ShiftedList() {
    delete[] array;
  }
  T getAt(int i) {
    return array[convertIndex(i)];
  }
  ...
};
```

Python

Classes

```
class Student:
  schoolName = 'Hanyang' #class attribute
  def __init__(self, id, name):
    self.id = id # instance attribute
    self.name = name # instance attribute



if __name__ == '__main__':
  std = Student(1011, "Sam")
  print(std.schoolName) // Hanyang
  print(std.name) // Sam
```

- Constructor __init__() is invoked automatically whenever a new object of a class is instantiated.

- By default, all data members and methods are public in Python.
  = Any member can be accessed from outside the class.

Classes

```
class Student:
  _schoolName = 'Hanyang’ # protected
  def __init__(self, id, name):
    self.__id = id # private attr
    self._name = name # protected attr

if __name__ == '__main__’:
  std = Student(1011, "Sam")
  print(std._schoolName) // Hanyang
  print(std._Student__id) // 1011
```

- Protected: add a prefix _ (single underscore)
- Private: add double underscore prefixed to a variable name

- Just recommendation for developers
- Conceptual implementation
  - They do not prevent instance variables from accessing or modifying the instance.

Inheritance

```
class Person:
  def __init__(self, i, n):
    self.id = i
    self.name = n

  def aboutMe(self):
    print("I am a person.")
```

```
class Student(Person):
  def __init__(self, i, n):
    super().__init__(i, n)

  def aboutMe(self):
    print("I am a student.")
```

Simple Syntax

- No pointers or references
- No destructors: Python has garbage collection.
- No virtual functions: duck typing
- No templates: duck typing

- Duck typing
  - If it walks like a duck and it quacks like a duck, then it must be a duck
  - In duck typing, an object is of a given type if it has all methods and properties required by that type. In contrast, with nominative typing, an object is of a given type if it is declared to be.